

std::vector

在标头 <vector> 定义

```
template<
    class T,
    class Allocator = std::allocator<T>
> class vector;

namespace pmr {
    template< class T >
    using vector = std::vector<T, std::pmr::polymorphic_allocator<T>>;
}
```

- 1) std::vector 是封装动态数组的序列容器。
- 2) std::pmr::vector 是使用多态分配器的模板别名。

元素被连续存储，这意味着不仅可通过迭代器，还能用指向元素的常规指针访问元素。这意味着指向 vector 元素的指针能传递给任何期待指向数组元素的指针的函数。

vector 的存储是自动管理的，按需扩张收缩。vector 通常占用多于静态数组的空间，因为要分配更多内存以管理将来的增长。vector 所用的方式不在每次插入元素时，而只在额外内存耗尽时进行重分配。分配的内存总量可用 capacity() 函数查询。可以通过调用 shrink_to_fit()^[1] 返回多余的内存给系统。

重分配通常是性能上有开销的操作。如果元素数量已知，那么 reserve() 函数可用于消除重分配。

vector 上的常见操作复杂度（效率）如下：

- 随机访问——常数 $O(1)$ 。
- 在末尾插入或删除元素——均摊常数 $O(1)$ 。
- 插入或删除元素——与到 vector 结尾的距离成线性 $O(n)$ 。

std::vector (对于 bool 以外的 T) 满足容器 (Container)、知分配器容器 (AllocatorAwareContainer)、序列容器 (SequenceContainer)、连续容器 (ContiguousContainer) (C++17 起) 及可逆容器 (ReversibleContainer) 的要求。

std::vector 的成员函数都是 constexpr 的：可以在常量表达式的求值中创建并使用 std::vector 对象。

然而 std::vector 对象通常不能是 constexpr 的，因为任何动态分配的存储都必须在同一常量表达式求值中释放。

1. ↑ 在 libstdc++ 中，shrink_to_fit() 不能 (<https://gcc.gnu.org/onlinedocs/libstdc++/manual/strings.html#strings.string.shrink>) 在 C++98 模式中使用。

模板形参

T - 元素的类型。

T 必须满足可复制赋值 (CopyAssignable) 和可复制构造 (CopyConstructible) 的要求。 (C++11 前)

加诸元素的要求依赖于容器上进行的实际操作。泛言之，要求元素类型是完整类型并满足可擦除 (Erasable) 的要求，但许多成员函数附带了更严格的要求。 (C++11 起) (C++17 前)

加诸元素的要求依赖于容器上进行的实际操作。泛言之，要求元素类型满足可擦除 (Erasable) 的要求，但许多成员函数附带了更严格的要求。如果分配器满足分配器完整性要求，那么容器（注意不是容器成员）能以不完整的元素类型实例化。

功能特性测试宏	值	标准	功能特性
__cpp_lib_incomplete_container_elements	201505L	(C++17)	最低限度支持不完整类型

 (C++17 起)

Allocator - 用于获取/释放内存及构造/析构内存中元素的分配器。类型必须满足分配器 (Allocator) 的要求。如果 Allocator::value_type 与 T 不同，那么行为未定义 (C++20 前) 程序非良构 (C++20 起)。

特化

标准库提供 `std::vector` 对类型 `bool` 的特化，它可能为空间效率优化。

`vector<bool>` 节省空间的动态 bitset
(类模板特化)

迭代器失效

操作	失效
所有只读操作	决不
<code>swap</code> 、 <code>std::swap</code>	<code>end()</code>
<code>clear</code> 、 <code>operator=</code> 、 <code>assign</code>	始终
<code>reserve</code> 、 <code>shrink_to_fit</code>	<code>vector</code> 更改容量时全部失效。否则不失效。
<code>erase</code>	被擦除元素及之后的所有元素（包括 <code>end()</code> ）。
<code>push_back</code> 、 <code>emplace_back</code>	<code>vector</code> 更改容量时全部失效。否则只有 <code>end()</code> 。
<code>insert</code> 、 <code>emplace</code>	<code>vector</code> 更改容量时全部失效。否则只有在或于插入点后者（包括 <code>end()</code> ）。
<code>resize</code>	<code>vector</code> 更改容量时全部失效。否则只有 <code>end()</code> 与被擦除元素。
<code>pop_back</code>	被擦除元素和 <code>end()</code> 。

成员类型

成员类型	定义
<code>value_type</code>	<code>T</code>
<code>allocator_type</code>	<code>Allocator</code>
<code>size_type</code>	无符号整数类型（通常是 <code>std::size_t</code> ）
<code>difference_type</code>	有符号整数类型（通常是 <code>std::ptrdiff_t</code> ）
<code>reference</code>	<code>value_type&</code>
<code>const_reference</code>	<code>const value_type&</code>
<code>pointer</code>	<code>Allocator::pointer</code> (C++11 前) <code>std::allocator_traits<Allocator>::pointer</code> (C++11 起)
<code>const_pointer</code>	<code>Allocator::const_pointer</code> (C++11 前) <code>std::allocator_traits<Allocator>::const_pointer</code> (C++11 起)
<code>iterator</code>	指向 <code>value_type</code> 的老式随机访问迭代器 (<i>LegacyRandomAccessIterator</i>) 及老式连续迭代器 (<i>LegacyContiguousIterator</i>) (C++20 前) 指向 <code>value_type</code> 的老式随机访问迭代器 (<i>LegacyRandomAccessIterator</i>)、 <code>contiguous_iterator</code> 及常量表达式迭代器 (<i>ConstexprIterator</i>) (C++20 起)
<code>const_iterator</code>	指向 <code>const value_type</code> 的老式随机访问迭代器 (<i>LegacyRandomAccessIterator</i>) 及老式连续迭代器 (<i>LegacyContiguousIterator</i>) (C++20 前) 指向 <code>const value_type</code> 的老式随机访问迭代器 (<i>LegacyRandomAccessIterator</i>)、 <code>contiguous_iterator</code> 及常量表达式迭代器 (<i>ConstexprIterator</i>) (C++20 起)
<code>reverse_iterator</code>	<code>std::reverse_iterator<iterator></code>
<code>const_reverse_iterator</code>	<code>std::reverse_iterator<const_iterator></code>

成员函数

(构造函数)	构造 <code>vector</code> (公开成员函数)
(析构函数)	析构 <code>vector</code> (公开成员函数)
<code>operator=</code>	赋值给容器 (公开成员函数)
<code>assign</code>	将值赋给容器 (公开成员函数)
<code>assign_range</code> (C++23)	将一个范围的值赋给容器 (公开成员函数)
<code>get_allocator</code>	返回相关的分配器 (公开成员函数)

元素访问

at	访问指定的元素，同时进行越界检查 (公开成员函数)
operator[]	访问指定的元素 (公开成员函数)
front	访问第一个元素 (公开成员函数)
back	访问最后一个元素 (公开成员函数)
data	直接访问底层数组 (公开成员函数)

迭代器

begin cbegin (C++11)	返回指向起始的迭代器 (公开成员函数)
end cend (C++11)	返回指向末尾的迭代器 (公开成员函数)
rbegin crbegin (C++11)	返回指向起始的逆向迭代器 (公开成员函数)
rend crend (C++11)	返回指向末尾的逆向迭代器 (公开成员函数)

容量

empty	检查容器是否为空 (公开成员函数)
size	返回容纳的元素数 (公开成员函数)
max_size	返回可容纳的最大元素数 (公开成员函数)
reserve	预留存储空间 (公开成员函数)
capacity	返回当前存储空间能够容纳的元素数 (公开成员函数)
shrink_to_fit	通过释放未使用的内存减少内存的使用 (公开成员函数)

修改器

clear	清除内容 (公开成员函数)
insert	插入元素 (公开成员函数)
insert_range (C++23)	插入元素的范围 (公开成员函数)
emplace (C++11)	原位构造元素 (公开成员函数)
erase	擦除元素 (公开成员函数)
push_back	将元素添加到容器末尾 (公开成员函数)
emplace_back (C++11)	在容器末尾原位构造元素 (公开成员函数)
append_range (C++23)	添加元素的范围到末尾 (公开成员函数)
pop_back	移除末元素 (公开成员函数)
resize	改变存储元素的个数 (公开成员函数)
swap	交换内容 (公开成员函数)

非成员函数

operator==

operator!=

operator<

operator<=

operator>

operator>=

operator<=>

(C++20 中移除)

(C++20 中移除)

(C++20 中移除)

(C++20 中移除)

(C++20 中移除)

(C++20 中移除)

(C++20)

按照字典顺序比较 vector 中的值
(函数模板)

std::swap

(std::vector)

特化 std::swap 算法
(函数模板)

erase

(std::vector)

erase_if

(std::vector)

(C++20)

擦除所有满足特定判别标准的元素
(函数模板)

推导指引 (C++17 起)

注解

功能特性测试宏	值	标准	功能特性
__cpp_lib_containers_ranges	202202L	(C++23)	容器的范围构造和插入

示例

运行此代码

```
#include <iostream>
#include <vector>

int main()
{
    // 创建含有整数的 vector
    std::vector<int> v = {8, 4, 5, 9};

    // 将两个整数添加到 vector
    v.push_back(6);
    v.push_back(9);

    // 覆写位置 2 的元素
    v[2] = -1;

    // 打印 vector 的值
    for (int n : v)
        std::cout << n << ' ';
    std::cout << '\n';
}
```

输出：

8 4 -1 9 6 9

缺陷报告

下列更改行为的缺陷报告追溯地应用于以前出版的 C++ 标准。

缺陷报告	应用于	出版时的行为	正确行为
LWG 69 (https://cplusplus.github.io/LWG/issue69)	C++98	未要求 vector 的元素存储的连续性	已要求

LWG 230 (https://cplusplus.github.io/LWG/issue230)	C++98	T 不需要满足可复制构造 (<i>CopyConstructible</i>) (因此不保证能构造 T 类型的元素)	T 也需要满足可复制构造 (<i>CopyConstructible</i>)
LWG 464 (https://cplusplus.github.io/LWG/issue464)	C++98	访问空的 <code>vector</code> 的底层存储导致未定义行为	提供 <code>data</code> 函数

来自“<https://zh.cppreference.com/mwiki/index.php?title=cpp/container/vector&oldid=82396>”